

MYSQL ASSIGNMENT 2 - QUERYING DATA

SCREENSHOTS

TASKS:

CLAUSE & OPERATORS

1. DISTINCT VALUES:

- A query to retrieve distinct salaries from the Employees table.

SCREENSHOT:

```
89      /* Clause & Operators: */
90
91      /*Distinct Clause*/
92
93 •    select distinct salary from employees;
94
95
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	salary
▶	60000.00
	80000.00
	45000.00
	70000.00
	55000.00
	65000.00
	90000.00
	95000.00
	75000.00
	85000.00
	100000.00

2. ALIAS (AS):

- Provide aliases for the "age" and "salary" columns as "Employee_Age" and "Employee_Salary", respectively.

SCREENSHOT OF EMPLOYEE_AGE:

Query 1 Employee Database S2* x

Limit to 1000 rows

```
96 /*Distinct Clause*/
97
98 • select distinct salary from employees;
99
100 /*Alias (as)*/
101
102 • select age as Employee_age from employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Employee_age
27
31
26
29
25
28
32
27
30
29
28
26
33
30
27
31
28
25

employees 5 x

SCREENSHOT OF EMPLOYEE_SALARY:

Query 1 Employee Database SZ x

Limit to 1000 rows

```

99
100  /*Alias (as)*/
101
102 •  select age as Employee_age from employees;
103
104 •  select salary as Employee_salary from employees;
105

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	Employee_salary
▶	60000.00
	80000.00
	45000.00
	70000.00
	55000.00
	65000.00
	90000.00
	70000.00
	55000.00
	80000.00
	60000.00
	95000.00
	75000.00
	60000.00
	85000.00
	100000.00
	80000.00
	75000.00
	55000.00

3. WHERE CLAUSE & OPERATORS:

- Retrieve employees with a salary greater than ₹50000 and hired before 2016-01-01.

SCREENSHOT:

Employee Database S2*

Limit to 1000 rows

```

101
102 • select age as Employee_age from employees;
103
104                               /*WHERE CLAUSE & OPERATORS: */
105 -- where & and operator--
106
107 • select * from employees where salary >50000 and hire_date <'2016-01-01';

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: IA

employee_id	employee_name	gender	age	hire_date	designation	salary	department_id	location_id
5001	Vihaan Singh	M	27	2015-01-20	Data Analyst	60000.00	3	4
5002	Reyansh Singh	M	31	2015-03-10	Network Engineer	80000.00	12	1
5004	Kiara Malhotra	F	29	2015-07-05	NULL	70000.00	8	3
5005	Anvi Chaudhary	F	25	2015-09-11	Business Development Executive	55000.00	11	1
5006	Dhruv Shetty	M	28	2015-11-20	UI Developer	65000.00	8	2
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Find the employee whose designation is missing and fill it with "Data Scientist".

SCREENSHOT:

```

110
111 • select * from employees where designation is null;
112
113 • update employees set designation = "Data Scientist" where designation is null;
114
115 • select * from employees where designation = "Data Scientist";

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: IA

employee_id	employee_name	gender	age	hire_date	designation	salary	department_id	location_id
5004	Kiara Malhotra	F	29	2015-07-05	Data Scientist	70000.00	8	3
5025	Avni Iyengar	F	31	2019-01-20	Data Scientist	100000.00	3	4
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SORTING AND GROUPING DATA

1. ORDER BY:

- Find employees sorted by department ID in ascending order and salary in descending order.

SCREENSHOT:

```

120
121                                     /* Sorting and Grouping Data */
122     /* order by */
123
124 • select * from employees order by department_id, salary desc;
125

```

Result Grid		Filter Rows:			Edit:		Export/Import:		Wrap Cell Content:	
	employee_id	employee_name	gender	age	hire_date	designation	salary	department_id	location_id	
▶	5020	Aarav Nair	M	32	2018-03-05	Software Engineer	90000.00	1	1	
	5007	Anushka Singh	F	32	2016-01-15	Marketing Manager	90000.00	2	3	
	5023	Anika Paul	F	30	2018-09-20	Public Relations Specialist	70000.00	2	2	
	5011	Saanvi Patel	F	28	2016-09-20	Marketing Analyst	60000.00	2	1	
	5025	Avni Iyengar	F	31	2019-01-20	Data Scientist	100000.00	3	4	
	5026	Vivaan Singh	M	29	2019-03-10	Business Analyst	75000.00	3	2	
	5001	Vihaan Singh	M	27	2015-01-20	Data Analyst	60000.00	3	4	
	5016	Zoya Khan	F	31	2017-07-05	Legal Counsel	100000.00	4	4	
	5022	Aadhya Iyengar	F	28	2018-07-10	HR Specialist	60000.00	4	4	
	5019	Ishika Patel	F	29	2018-01-15	Talent Acquisition Specialist	55000.00	4	4	
	5024	Aryan Shetty	M	27	2018-11-05	Product Manager	95000.00	5	1	
	5030	Sara Iyer	F	28	2019-11-20	Project Manager	90000.00	5	1	
	5027	Ananya Paul	F	32	2019-05-20	Content Writer	60000.00	6	3	
	5028	Anaya Kapoor	F	26	2019-07-05	Event Coordinator	60000.00	6	1	
	5015	Ishaan Kumar	M	27	2017-05-20	Financial Analyst	85000.00	7	1	
	5021	Advik Kapoor	M	26	2018-05-20	Finance Analyst	85000.00	7	3	
	5004	Kiara Malhotra	F	29	2015-07-05	Data Scientist	70000.00	8	3	
	5008	Diya Jha	F	27	2016-03-05	Graphic Designer	70000.00	8	4	

2. LIMIT:

- Display the first 5 employees hired in the year 2018.

SCREENSHOT:

126

/* limit */

127

128 •

select * from employees where hire_date < '2018-01-01' limit 5;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

Fetch rows:

employee_id	employee_name	gender	age	hire_date	designation	salary	department_id	location_id
5001	Vihaan Singh	M	27	2015-01-20	Data Analyst	60000.00	3	4
5002	Reyansh Singh	M	31	2015-03-10	Network Engineer	80000.00	12	1
5003	Aaradhya Iyer	F	26	2015-05-20	Customer Support Executive	45000.00	10	2
5004	Kiara Malhotra	F	29	2015-07-05	Data Scientist	70000.00	8	3
5005	Anvi Chaudhary	F	25	2015-09-11	Business Development Executive	55000.00	11	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

3. AGGREGATE FUNCTIONS:

- Calculate the sum of all salaries in the Finance department.

SCREENSHOT:

```

129
130      /* AGGREGATE FUNCTIONS  sum & min */
131
132 •      select sum(salary) as total_finace_salary
133      from employees
134      where department_id="7";

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_finace_salary			
▶	170000.00			

- Find the minimum age among all employees.

SCREENSHOT:

```

136 •      select min(age) as min_age
137      from employees;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	min_age			
▶	25			

4. GROUP BY:

- List the maximum salary for each location.

SCREENSHOT:

```

139      /* GROUP BY */
140
141 •    select location_id,
142      max(salary) as max_salary
143      from employees
144      group by location_id;

```

Result Grid			Filter Rows:	Export:
	location_id	max_salary		
▶	1	95000.00		
	2	95000.00		
	3	90000.00		
	4	100000.00		

- Calculate the average salary for each designation containing the word 'Analyst'.

SCREENSHOT:

```

146      /* Average*/
147
148 •    select designation,
149      avg(salary) as avg_salary
150      from employees
151      where designation like '%Analyst%'
152      group by designation;

```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	designation	avg_salary			
▶	Data Analyst	60000.000000			
	Marketing Analyst	60000.000000			
	Supply Chain Analyst	60000.000000			
	Financial Analyst	85000.000000			
	Finance Analyst	85000.000000			
	Business Analyst	75000.000000			
	Quality Assurance Analyst	80000.000000			

5. HAVING:

- Find departments with less than 3 employees.

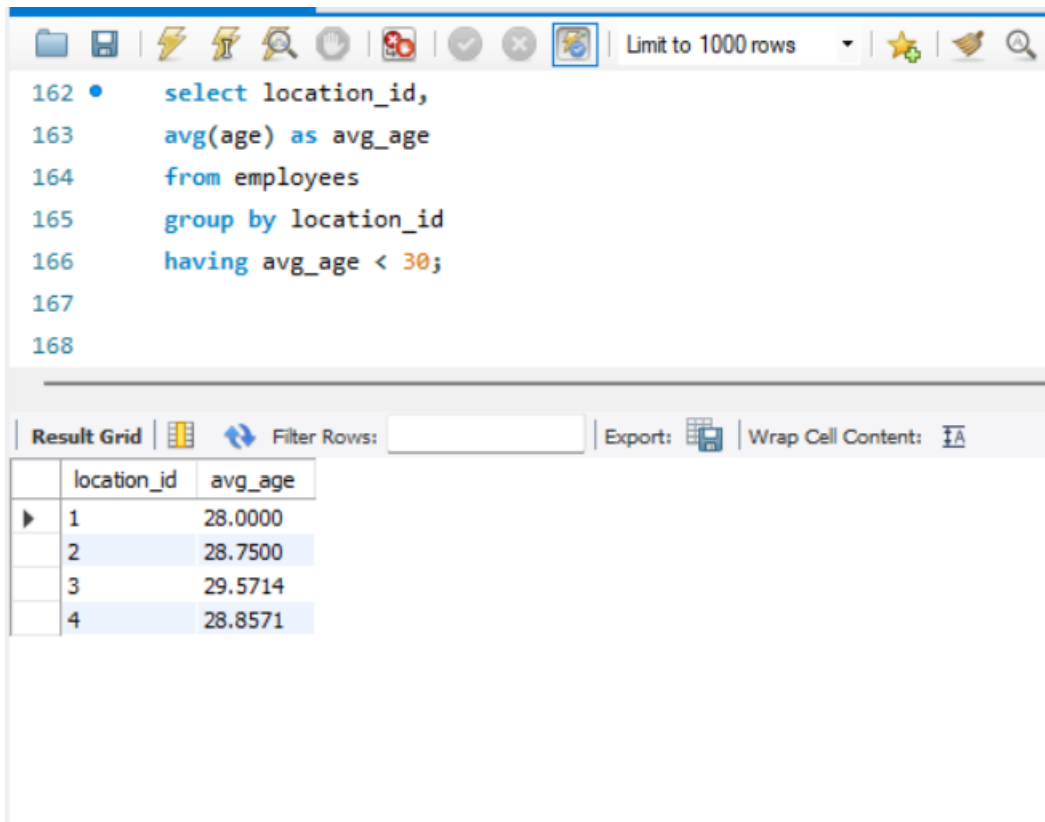
SCREENSHOT:

```
154      /* Having */
155
156 •   select department_id,
157      count(*) as employee_count
158   from employees
159   group by department_id
160   having COUNT(*) < 3;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
department_id	employee_count		
1	1		
5	2		
6	2		
7	2		
9	1		
11	2		
13	1		

- Find locations with female employees whose average age is below 30.

SCREENSHOT:



162 • `select location_id,`
163 `avg(age) as avg_age`
164 `from employees`
165 `group by location_id`
166 `having avg_age < 30;`
167
168

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	location_id	avg_age
▶	1	28.0000
	2	28.7500
	3	29.5714
	4	28.8571

JOINS

1. INNER JOIN:

- List employee names, their designations, and department names where employees are assigned to a department.

SCREENSHOT:

```

170      /* INNER JOIN */
171 •    select e.employee_name,e.designation,d.department_name
172      from employees as e
173      inner join departments as d on e.department_id=d.department_id;
174

```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 			
	employee_name	designation	department_name
•	Anvi Chaudhary	Business Development Executive	Business Development
	Kiaan Desai	Sales Executive	Business Development
	Ananya Paul	Content Writer	Content Creation
	Anaya Kapoor	Event Coordinator	Content Creation
	Aaradhya Iyer	Customer Support Executive	Customer Support
	Arnav Rao	Customer Success Manager	Customer Support
	Vihaan Mohan	Supply Chain Analyst	Customer Support
	Vihaan Singh	Data Analyst	Data Science
	Avni Iyengar	Data Scientist	Data Science
	Vivaan Singh	Business Analyst	Data Science
	Kiara Malhotra	Data Scientist	Design
	Dhruv Shetty	UI Developer	Design
	Diya Jha	Graphic Designer	Design
	Ishaan Kumar	Financial Analyst	Finance
	Advik Kapoor	Finance Analyst	Finance
	Zoya Khan	Legal Counsel	Human Resources
	Ishika Patel	Talent Acquisition Specialist	Human Resources
	Aadhya Iyengar	HR Specialist	Human Resources

2. LEFT JOIN:

- List all departments along with the total number of employees in each department, including departments with no employees.

SCREENSHOT:

```

175      /* LEFT JOIN */
176 •    select d.department_name,
177      count(e.employee_id) as total_employees
178      from departments d
179      left join employees e
180      on d.department_id = e.department_id
181      group by d.department_name;

```

Result Grid		
	Filter Rows:	Export: Wn
department_name	total_employees	
Business Development	2	
Content Creation	2	
Customer Support	3	
Data Science	3	
Design	3	
Finance	2	
Human Resources	3	
IT	4	
Marketing	3	
Operations	1	
Product Management	2	
Research and Develo...	1	
Software Development	1	

3. RIGHT JOIN:

- Display all locations along with the names of employees assigned to each location. If no employees are assigned to a location, display NULL for employee name.

SCREENSHOT:

Employee Database S2*

Limit to 1000 rows

```

182
183  /* RIGHT JOIN */
184  • select l.location_name,e.employee_name from employees e
185  right join location l
186  on e.location_id = l.location_id;
187
188

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [\[A\]](#)

	location_name	employee_name
▶	Bangalore	Aaradhya Iyer
	Bangalore	Dhruv Shetty
	Bangalore	Myra Verma
	Bangalore	Vihaan Mohan
	Bangalore	Kabir Nair
	Bangalore	Anika Paul
	Bangalore	Vivaan Singh
	Bangalore	Arjun Kumar
	Chennai	Reyansh Singh
	Chennai	Anvi Chaudhary
	Chennai	Saanvi Patel
	Chennai	Ishaan Kumar
	Chennai	Aarav Nair
	Chennai	Aryan Shetty
	Chennai	Anaya Kapoor
	Chennai	Sara Iyer
	Hyderabad	Kiara Malhotra
	Hyderabad	Anushka Singh

4. CROSS JOIN

- Show all possible combinations of departments and locations.

SCREENSHOT:

```

191      /* CROSS JOIN */
192
193 •   select d.department_name,l.location_name from departments d
194      cross join location l;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	department_name	location_name		
▶	Business Development	Pune		
	Business Development	Hyderabad		
	Business Development	Chennai		
	Business Development	Bangalore		
	Content Creation	Pune		
	Content Creation	Hyderabad		
	Content Creation	Chennai		
	Content Creation	Bangalore		
	Customer Support	Pune		
	Customer Support	Hyderabad		
	Customer Support	Chennai		
	Customer Support	Bangalore		
	Data Science	Pune		
	Data Science	Hyderabad		
	Data Science	Chennai		
	Data Science	Bangalore		
	Design	Pune		
	Design	Hyderabad		

5. SELF JOIN:

- Show pairs of employees working in the same department, excluding self-pairs.

SCREENSHOT:

```

199 • select e1.employee_name as employee1,
200         e2.employee_name as employee2,
201         e1.department_id
202 from employees e1
203 join employees e2
204 on e1.department_id = e2.department_id
205 and e1.employee_id < e2.employee_id;

```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	employee1	employee2	department_id
▶	Vihaan Singh	Avni Iyengar	3
	Vihaan Singh	Vivaan Singh	3
	Reyansh Singh	Atharv Yadav	12
	Reyansh Singh	Kabir Nair	12
	Reyansh Singh	Arjun Kumar	12
	Aaradhya Iyer	Arnav Rao	10
	Aaradhya Iyer	Vihaan Mohan	10
	Kiara Malhotra	Dhruv Shetty	8
	Kiara Malhotra	Diya Jha	8
	Anvi Chaudhary	Kiaan Desai	11
	Dhruv Shetty	Diya Jha	8
	Anushka Singh	Saanvi Patel	2
	Anushka Singh	Anika Paul	2
	Atharv Yadav	Kabir Nair	12
	Atharv Yadav	Arjun Kumar	12
	Saanvi Patel	Anika Paul	2
	Arnav Rao	Vihaan Mohan	10
	Ishaan Kumar	Advik Kapoor	7

WINDOWS FUNCTION

- Write a window function query to rank employees by salary using rank().

SCREENSHOT:

```

210 • select employee_id,employee_name,salary,
211        rank() over (order by salary desc) as salary_rank
212 from employees;

```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

employee_id	employee_name	salary	salary_rank
5016	Zoya Khan	100000.00	1
5025	Avni Iyengar	100000.00	1
5012	Myra Verma	95000.00	3
5024	Aryan Shetty	95000.00	3
5007	Anushka Singh	90000.00	5
5020	Aarav Nair	90000.00	5
5030	Sara Iyer	90000.00	5
5015	Ishaan Kumar	85000.00	8
5021	Advik Kapoor	85000.00	8
5002	Reyansh Singh	80000.00	10
5010	Atharv Yadav	80000.00	10
5017	Kabir Nair	80000.00	10
5029	Arjun Kumar	80000.00	10
5013	Arnav Rao	75000.00	14
5018	Ishan Mishra	75000.00	14
5026	Vivaan Singh	75000.00	14
5004	Kiara Malhotra	70000.00	17
5008	Diya Jha	70000.00	17

- Write a window function query to rank employees by salary within each department using DENSE RANK()

SCREENSHOT:

```

214      /* DENSE_RANK */
215
216 •    select employee_id,employee_name,department_id,salary,
217       dense_rank() over (partition by department_id order by salary desc) as salary_rank
218       from employees;
219

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 					
	employee_id	employee_name	department_id	salary	salary_rank
▶	5020	Aarav Nair	1	90000.00	1
	5007	Anushka Singh	2	90000.00	1
	5023	Anika Paul	2	70000.00	2
	5011	Saanvi Patel	2	60000.00	3
	5025	Avni Iyengar	3	100000.00	1
	5026	Vivaan Singh	3	75000.00	2
	5001	Vihaan Singh	3	60000.00	3
	5016	Zoya Khan	4	100000.00	1
	5022	Aadhya Iyengar	4	60000.00	2
	5019	Ishika Patel	4	55000.00	3
	5024	Aryan Shetty	5	95000.00	1
	5030	Sara Iyer	5	90000.00	2
	5027	Ananya Paul	6	60000.00	1
	5028	Anaya Kapoor	6	60000.00	1
	5015	Ishaan Kumar	7	85000.00	1
	5021	Advik Kapoor	7	85000.00	1
	5004	Kiara Malhotra	8	70000.00	1

- Write a window function query, Running total salary by department

SCREENSHOT:


```

220 • select employee_id,employee_name,department_id,salary,
221 SUM(salary) over (partition by department_id order by employee_id) as running_total_salary
222 from employees;

```

Result Grid				
Filter Rows:		Export:	Wrap Cell Content:	
employee_id	employee_name	department_id	salary	running_total_salary
5020	Aarav Nair	1	90000.00	90000.00
5007	Anushka Singh	2	90000.00	90000.00
5011	Saanvi Patel	2	60000.00	150000.00
5023	Anika Paul	2	70000.00	220000.00
5001	Vihaan Singh	3	60000.00	60000.00
5025	Avni Iyengar	3	100000.00	160000.00
5026	Vivaan Singh	3	75000.00	235000.00
5016	Zoya Khan	4	100000.00	100000.00
5019	Ishika Patel	4	55000.00	155000.00
5022	Aadhya Iyengar	4	60000.00	215000.00
5024	Aryan Shetty	5	95000.00	95000.00
5030	Sara Iyer	5	90000.00	185000.00
5027	Ananya Paul	6	60000.00	60000.00
5028	Anaya Kapoor	6	60000.00	120000.00
5015	Ishaan Kumar	7	85000.00	85000.00
5021	Advik Kapoor	7	85000.00	170000.00
5004	Kiara Malhotra	8	70000.00	70000.00